```
  GNU nano 7.2                          signal_handler.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig) {
    printf("\nSignal %d received!\n", sig);
}

int main(void) {
    signal(SIGINT, handle_signal);
    printf("Signal handler running. Press Ctrl+C.\n");

    while (1)
        pause();

    return 0;
}
```

```
krishnasree08@krishnasree08:~/ossp_session17_18$ mkdir -p ~/ossp_session19_2
0
cd ~/ossp_session19_20
krishnasree08@krishnasree08:~/ossp_session19_20$ signal_handler.c
signal_handler.c: command not found
krishnasree08@krishnasree08:~/ossp_session19_20$ nano signal_handler.c
krishnasree08@krishnasree08:~/ossp_session19_20$ gcc -Wall -Wextra signal_ha
ndler.c -o signal_handler
./signal_handler
Signal handler running. Press Ctrl+C.
^C
Signal 2 received!
```

```
  GNU nano 7.2                    sigint_forward.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child = -1;

void handle_int(int sig) {
    (void)sig;
    if (child > 0)
        kill(child, SIGINT);
}

int main(void) {
    signal(SIGINT, handle_int);

    child = fork();

    if (child == 0) {
        signal(SIGINT, SIG_DFL);
        while (1) {
            printf("Child process running...\n");
            sleep(1);
        }
    } else if (child > 0) {
        printf("Parent running. Press Ctrl+C to interrupt child.\n");
        waitpid(child, NULL, 0);
        printf("Child terminated. Parent continues.\n");
    } else {
        perror("fork");
        return 1;
    }

    return 0;
}
```

```
krishnasree08@krishnasree08:~$ nano sigint_forward.c
krishnasree08@krishnasree08:~$ gcc -Wall -Wextra sigint_forward.c -o sigint_
forward
./sigint_forward
Parent running. Press Ctrl+C to interrupt child.
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...

Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
Child process running...
```

Overall Result – Sessions 19 & 20

Thus, the programs were implemented to demonstrate signal handling, signal forwarding, process suspension, process group management, and parent–child process coordination using system calls and signal-handling functions. The experiments helped understand how operating systems manage signals, control processes, and coordinate process execution.